



Towards Real-Scene Image Restoration

Chao-Tsung Huang

**National Tsing Hua University
Department of Electrical Engineering**



Outline

- **On real-world dataset collection**
 - **RealBlur**
- On data augmentation
 - Creating realistic dataset
 - Learning noise characteristics
- On modeling
 - Zero-shot single-image super-resolution
 - Image-adaptive correction filter for super-resolution

Problems of synthetic dataset

- Unrealistic artifacts, like discontinuous blurs, appear in synthetic dataset, like GoPro



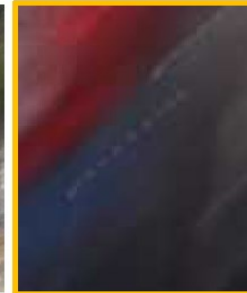
(a) GoPro dataset



(b) Magnified view of (a)



(c) GoPro dataset



(d) Magnified view of (c)



(e) Real-world low-light blurred image

- Trained models are good for synthetic test dataset but not good for real-world images



(a) Blurred Image

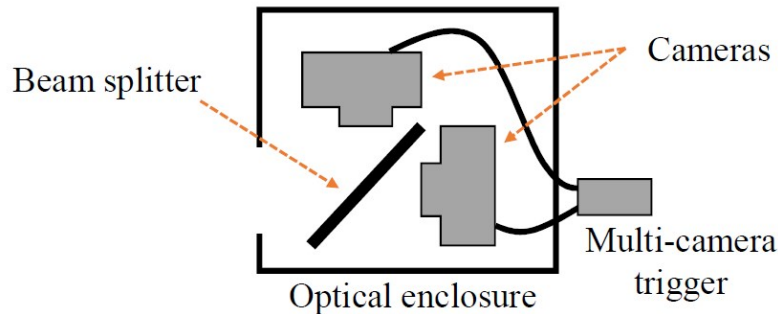


(d) SRN-DeblurNet [42]

[J. Rim et. al., ECCV 2020.]

RealBlur: real-world blur dataset

- Dual-camera system



(a) A diagram of our image acquisition system



(b) Our image acquisition system

- Post-processing
 - ISP, downsampling, geometric alignment, photometric alignment
- Datasets
 - RealBlur-R from RAW format
 - RealBlur-J from JPEG (using camera ISP)

Subjective evaluation for real pictures

- Gradual improvement by adding dataset



(a) Blurred Image



(d) SRN-DeblurNet [42]



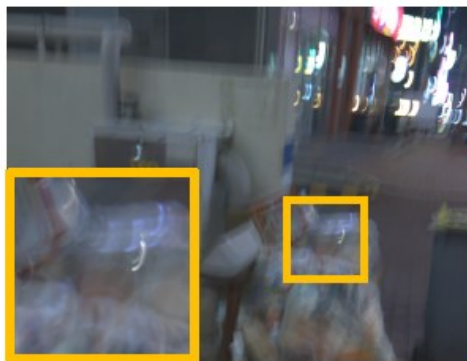
(g) SRN-DeblurNet [42]
RealBlur-J



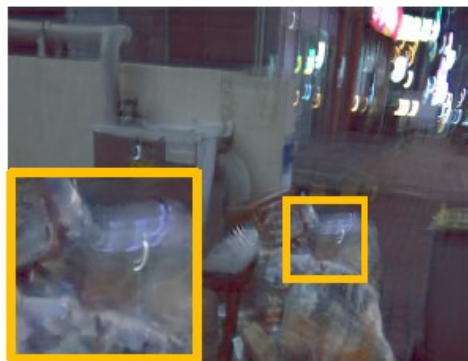
(h) SRN-DeblurNet [42]
RealBlur-J+GoPro+BSD-B

Subjective evaluation for real pictures

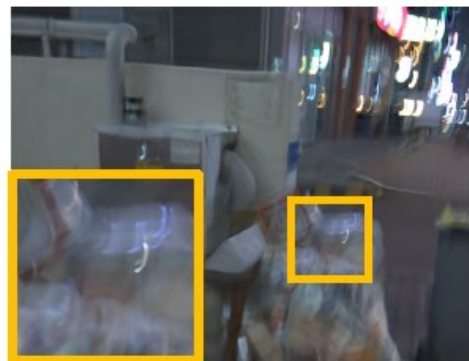
- *: trained w/ RealBlur dataset



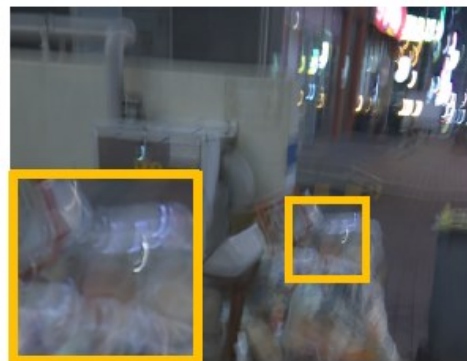
(k) Blurred image
PSNR/SSIM



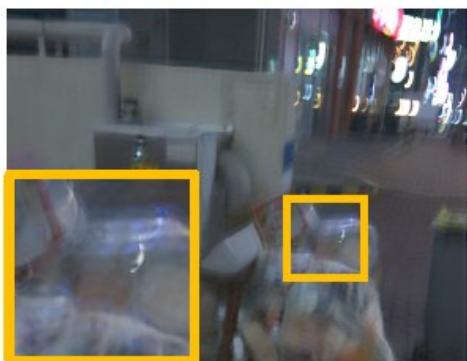
(l) Nah et al. [31]
23.37/0.8136



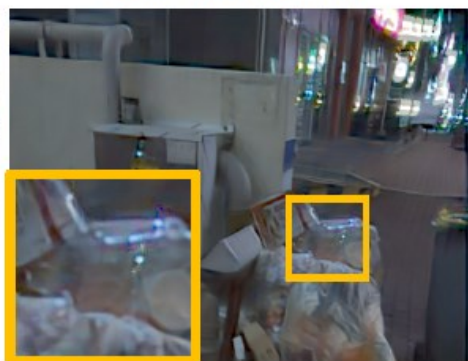
(m) Zhang et al. [49]
23.46/0.8354



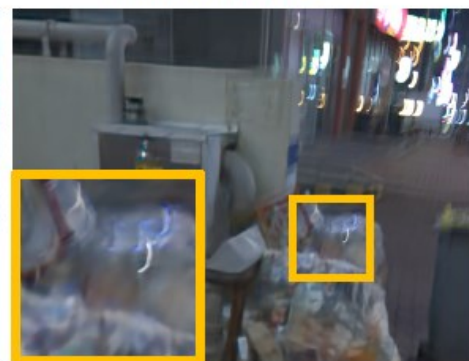
(n) Zhang et al. [48]
23.36/0.8155



(p) DeblurGAN-v2 [22]
23.47/0.8453



(q) DeblurGAN-v2* [22]
26.68/0.8988



(r) SRN-DeblurNet [42]
23.56/0.8507



(s) SRN-DeblurNet* [42]
29.42/0.9321



Outline

- On real-world dataset collection
- **On data augmentation**
 - **Creating realistic dataset**
 - RSBlur
 - Learning noise characteristics
- On modeling
 - Zero-shot single-image super-resolution
 - Image-adaptive correction filter for super-resolution

RSBlur: realistic blur synthesis

- Build the first dataset of real blurred images and multiple sharp images for analysis
- Analyze the difference between real and synthetic blurs
 - Identify issues of discontinuity, saturation, and noise



(a) Blurred image

(b) Discontinuity

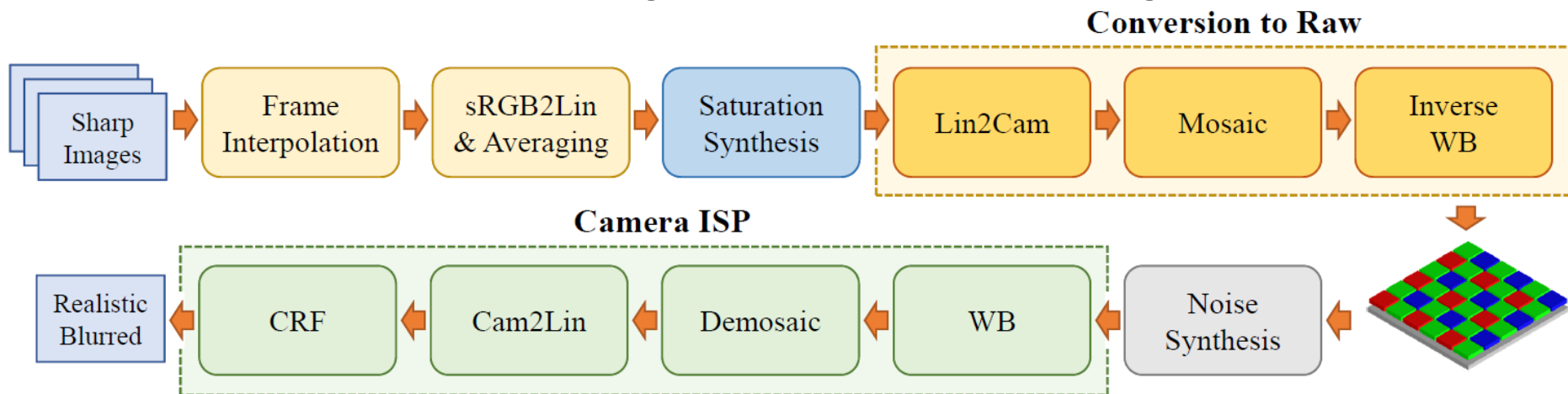
(c) Saturation

(d) Noise

[J. Rim et. al., ECCV 2022.]

Proposed synthesis pipeline

- Sharp image back to RAW
 - Saturation and noise injection in linear domain
- Then perform ISP to generate blurred image



Evaluation with different training datasets



(a) Blurred image
PSNR/SSIM

(b) Method 1
29.81/0.8066

(c) Method 5
26.87/0.7129

RSBlur $\leftarrow$ real blurred training set



(d) Method 6
27.45/0.7938

(e) Method 10
29.38/0.8027

(f) Ground truth

Ours $\leftarrow$ proposed synthesis pipeline

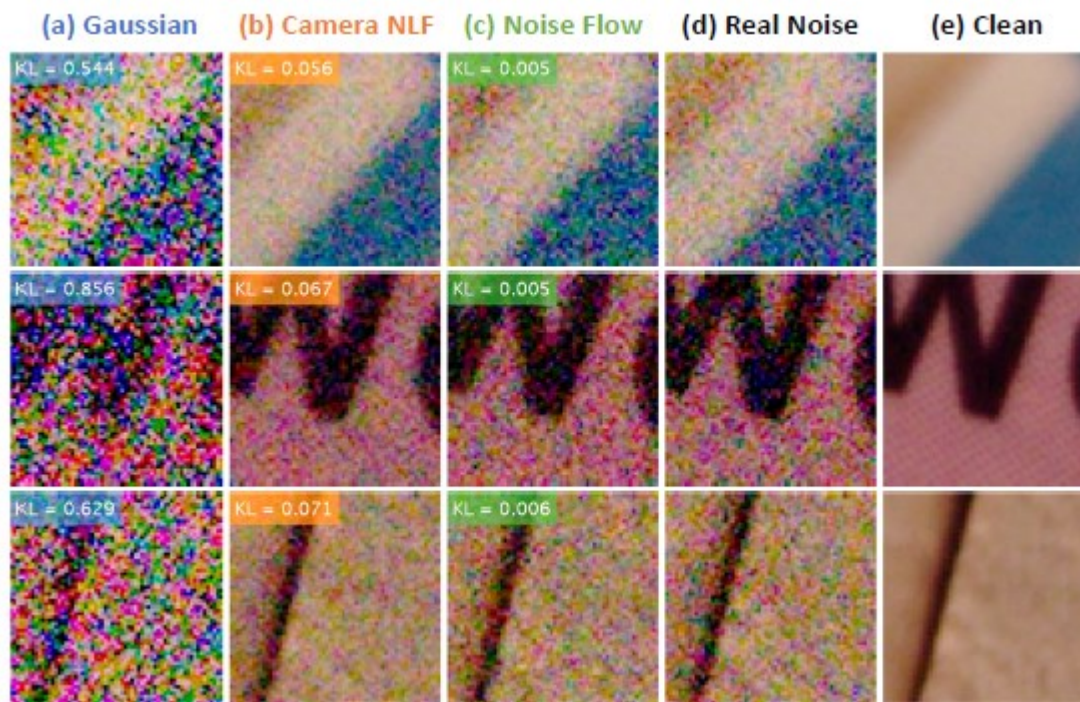


Outline

- On real-world dataset collection
- **On data augmentation**
 - Creating realistic dataset
 - **Learning noise characteristics**
 - **Noise flow**
 - Camera-aware model
- On modeling
 - Zero-shot single-image super-resolution
 - Image-adaptive correction filter for super-resolution

Noise Flow

- Learn noise model from real-world clean/noisy pairs
 - SIDD dataset: >1000 images, 10 scenes, five mobile phones, different ISO levels
- Then train CNN from massive synthetic clean/noisy pairs



KL divergence between generated and real noises (pixel marginals)

[A. Abdelhamed et. al., ICCV 2019.]



Background: Noise level function

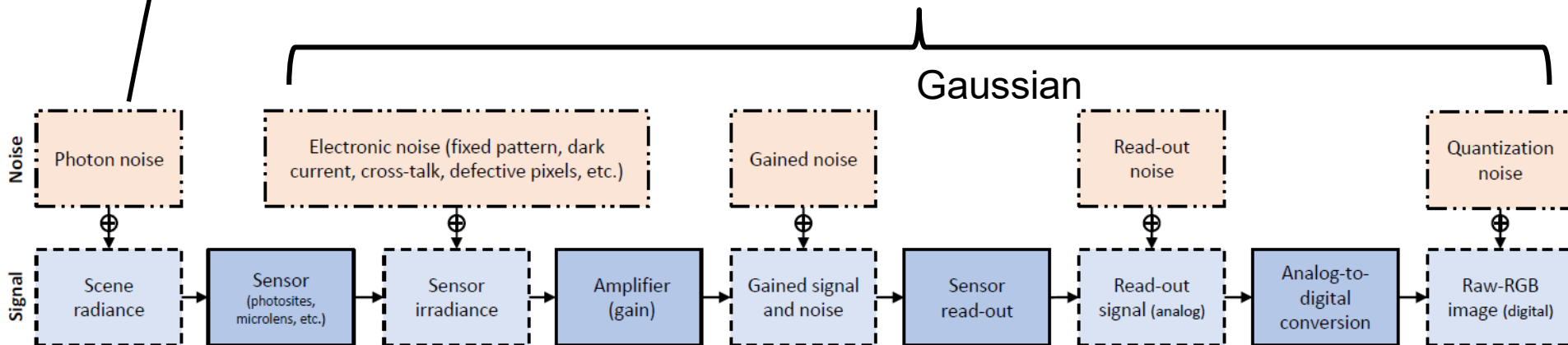
$$n_i \sim \alpha \mathcal{P}(\mathbf{I}_i) - \mathbf{I}_i$$

Poisson

$$n_i \sim \mathcal{N}(0, \alpha^2 \mathbf{I}_i + \delta^2)$$

Fail to capture real-scene details

$$n_i \sim \alpha \mathcal{P}(\mathbf{I}_i) - \mathbf{I}_i + \mathcal{N}(0, \delta^2)$$

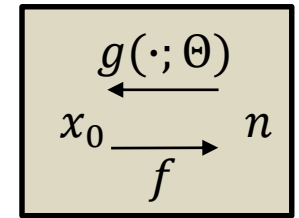




Background: Normalizing flows

- Density estimation

- Build bijective/invertible transform $g = f^{-1}$ between
 - Observable and to-be-learned variable n
 - Latent but tractable variable $x_0 = g(n)$



$$\mathbf{n} = f(\mathbf{x}_0) = f_N \circ f_{N-1} \circ \cdots \circ f_1(\mathbf{x}_0)$$

$g = g_1 \circ \cdots \circ g_{N-1} \circ g_N$ is the inverse of f

- Minimize negative log likelihood (NLL) on $p(n)$ to estimate parameterized g

$$p(\mathbf{n}) = p_{\mathcal{X}_0}(g(\mathbf{n})) \prod_{j=1}^N |\det \mathbf{J}_j(g(\mathbf{n}))|^{-1} \quad J_j = \frac{\partial f_j}{\partial x_{j-1}}$$

$$\sum_{i=1}^M \log p_{\mathcal{X}_0}(g(\mathbf{n}_i|\Theta)) - \sum_{j=1}^N \log |\det \mathbf{J}_j(g(\mathbf{n}_i|\Theta), \theta_j)|$$

- Synthetic noise sampling

- Sample tractable x_0 and have generated noise $n = f(x_0)$

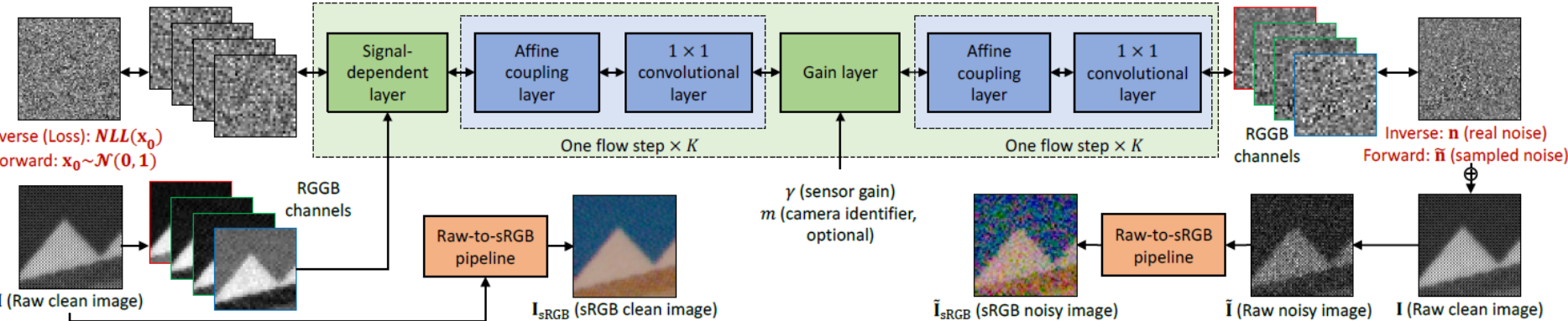
Noise Flow model and result

Hidden Gaussian noise

Forward (sampling) $\rightarrow$

Noise Flow

$\leftarrow$ Inverse (density estimation)



Auxiliary noise x_0

Training: Inverse flow, NLL evaluation
Testing: forward flow, sampled noise

	Gaussian	Cam. NLF	Noise Flow
NLL	-2.831 (99.4%)	-3.105 (51.6%)	-3.521
D_{KL}	0.394 (97.9%)	0.052 (84.1%)	0.008

Target noise n

Training: Inverse flow, real noise
Testing: forward flow, sampled noise

Camera identifier is treated as known info

Model	NLL	D_{KL}
S-G	-3.431 (9.42%)	0.067 (88.1%)
S-G-CAM	-3.511 (1.01%)	0.010 (20.0%)
S-Ax1-G-Ax1-CAM	-3.518 (0.30%)	0.009 (11.1%)
S-Ax4-G-Ax4-CAM (Noise Flow)	-3.521	0.008

On dataset On pixel marginals

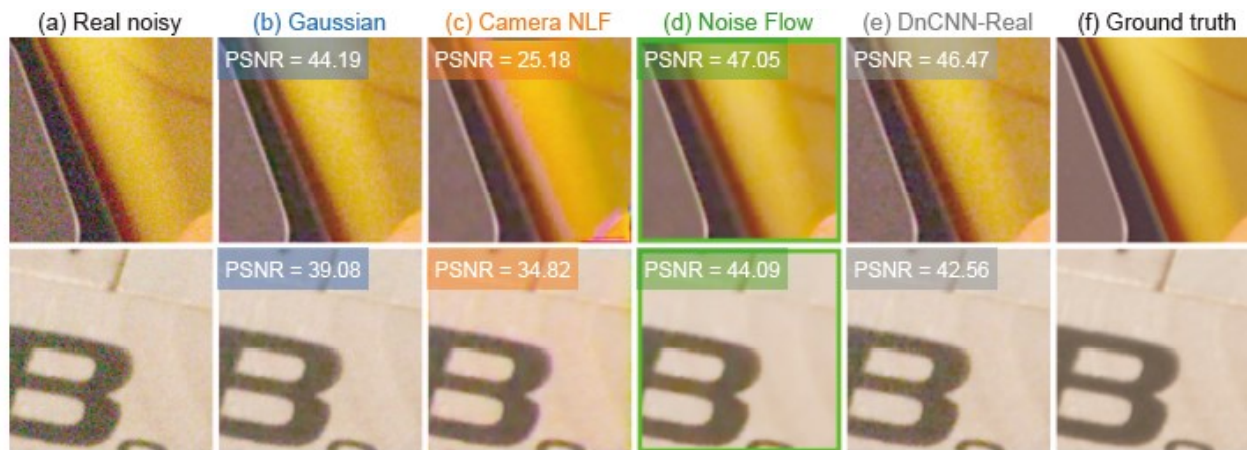
Signal-dependent and gain layers
Great improvement by camera-specific parameters

Real Image Denoising

- Train CNN using different noisy environment

Model	PSNR	SSIM	
DnCNN-Gauss	43.63 (43.0%)	0.968 (75.6%)	
DnCNN-CamNLF	44.99 (33.4%)	0.982 (56.0%)	
DnCNN-NF	48.52	0.992	
DnCNN-Real	47.08 (15.3%)	0.989 (27.5%)	(100k patches)
DnCNN-Real	48.30	0.994	(500k patches)

ref: K.-C. Chang et al., ECCV2020.

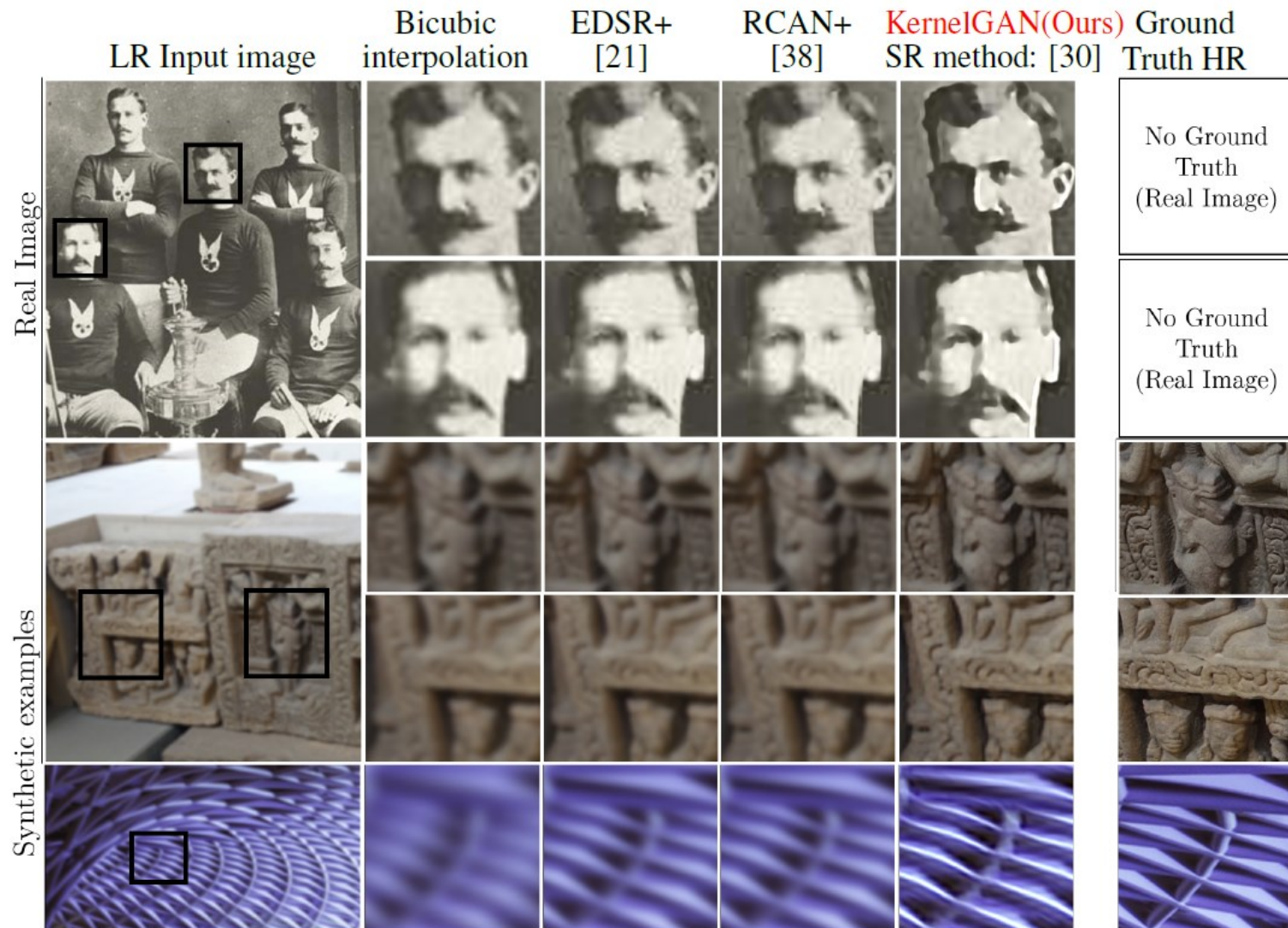




Outline

- On real-world dataset collection
- On data augmentation
 - Creating realistic dataset
 - Learning noise characteristics
- **On modeling**
 - **Zero-shot single-image super-resolution**
 - **KernelGAN (kernel estimation)+ZSSR (deconvolution)**
 - Image-adaptive correction filter for super-resolution

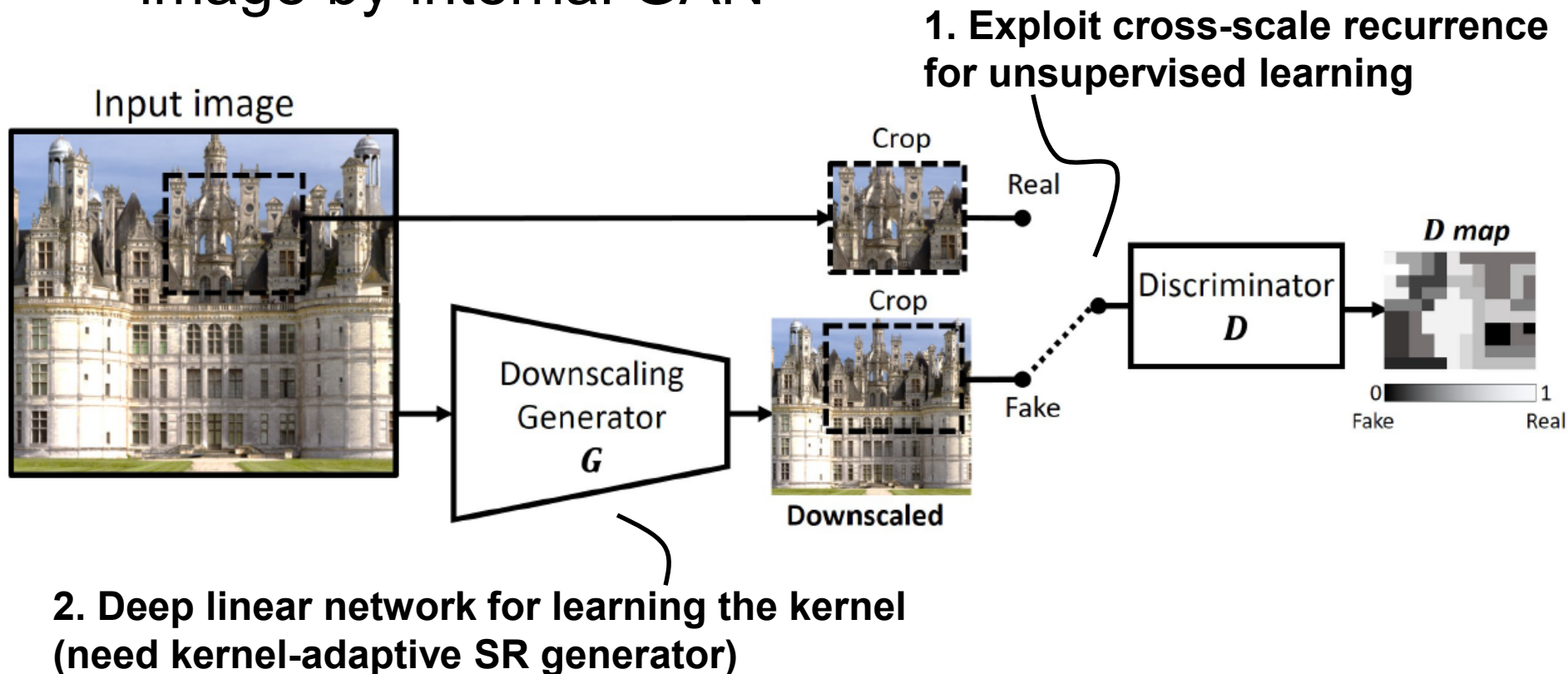
Issue: SR degradation for wrong kernel assumption



[S. Bell-Kligler et. al., NIPS, 2019.]

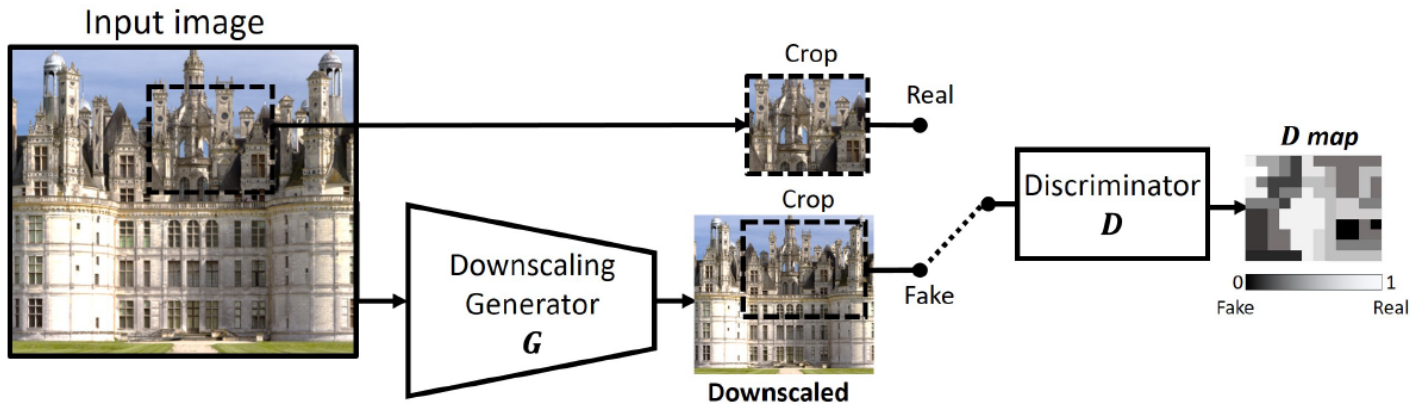
KernelGAN

- Learn image-specific kernel for low-resolution (LR) image by internal GAN



[S. Bell-Kligler et. al., NIPS 2019.]

1. Internal GAN



- D outputs a heat map
- LSGAN variant with L1 norm

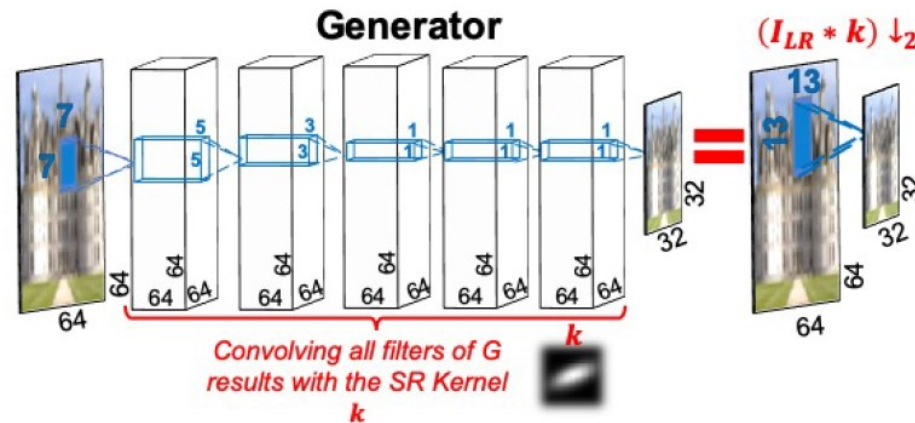
$$G^*(I_{LR}) = \operatorname{argmin}_G \max_D \{ \mathbb{E}_{x \sim \text{patches}(I_{LR})} [|D(x) - 1| + |D(G(x))|] + \mathcal{R} \}$$

- Learn X2 kernel only
 - X4 kernel is derived by $k_4 = k_2 * k_{2_dilated}$

$$I_{LR} * k_4 \downarrow_4 = (I_{LR} * k_2 \downarrow_2) * k_2 \downarrow_2$$

Assumption

2. Deep linear generator = SR kernel



- No non-linear layer (for Gaussian-like kernels)
- Better than single-layer generator

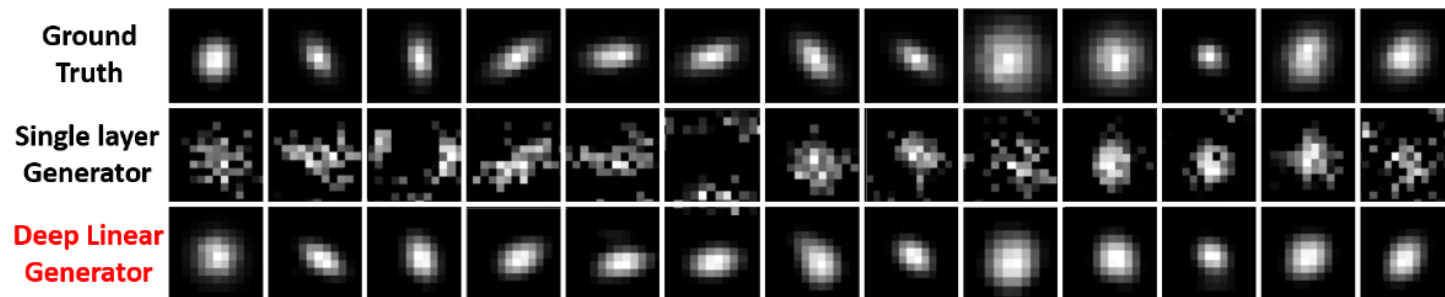
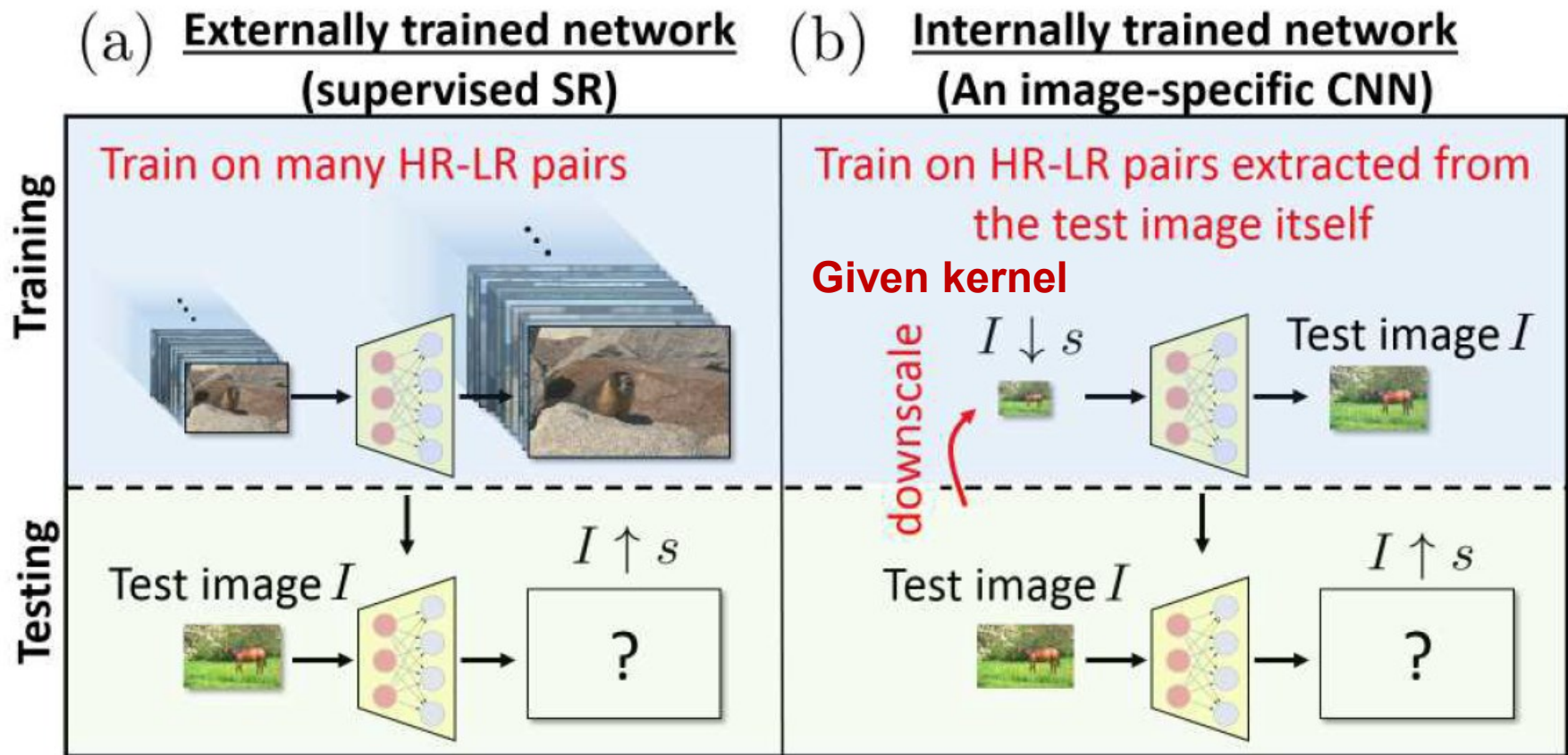


Image- and kernel-specific CNN generator

- Example: zero-shot super-resolution (ZSSR) using deep internal learning

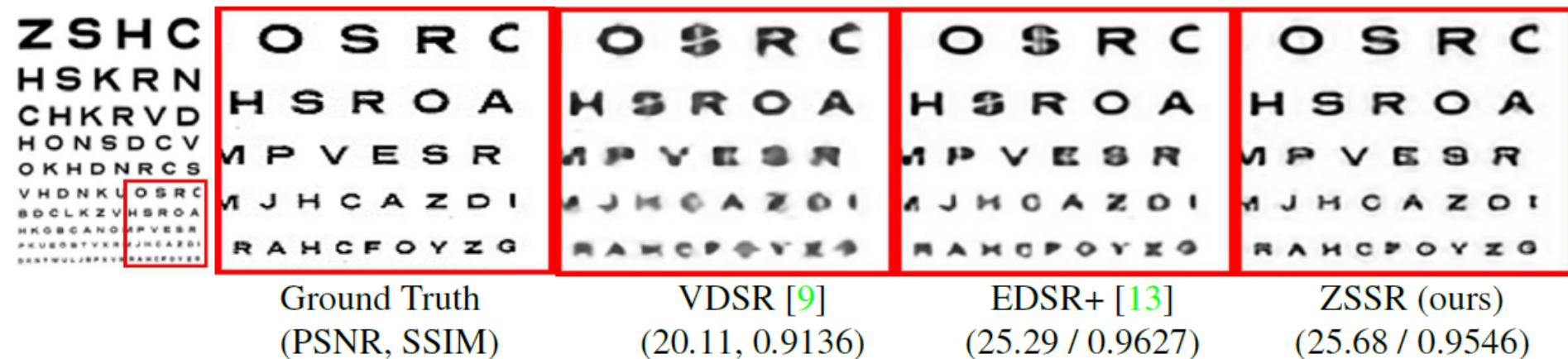


[A. Shocher et. al., CVPR 2018.]



Image- and kernel-specific CNN generator

- Example: zero-shot super-resolution (ZSSR) using deep internal learning
 - Small CNN (8-layer, 64-ch) for fitting to one image
 - Claimed quick training and testing
 - Self-supervised from downsampled-LR-and-LR pairs



[A. Shocher et. al., CVPR 2018.]

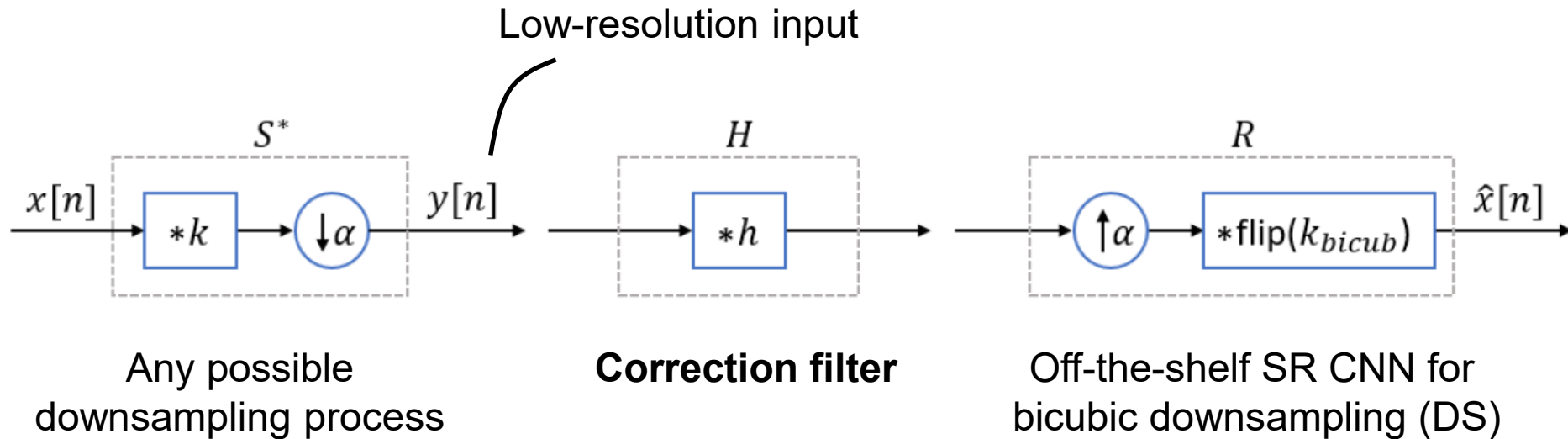


Outline

- On real-world dataset collection
- On data augmentation
 - Creating realistic dataset
 - Learning noise characteristics
- **On modeling**
 - Zero-shot single-image super-resolution
 - KernelGAN (kernel estimation)+ZSSR (deconvolution)
 - **Image-adaptive correction filter for super-resolution**



Correction Filter for Single Image Super-Resolution: Robustifying Off-the-Shelf Deep Super-Resolvers



Increase CNN generalization capability by using simple correction filters.

[S. Hussein et. al., CVPR 2020.]



R^* : bicubic DS;
 R : bicubic SR;
 S^* : current DS

Non-blind setting

Assume k is known

$$\begin{aligned}\hat{\mathbf{x}} &= \mathcal{R}\mathcal{H}\mathbf{y} \\ &= \mathcal{R}\mathcal{H}\mathcal{S}^*\mathbf{x} \\ &= \mathcal{R}\mathcal{H}\mathcal{S}^*\mathcal{R}\mathcal{R}^*\mathbf{x}\end{aligned}$$

$$\begin{aligned}\hat{\mathbf{x}} &= \mathcal{R}(\mathcal{S}^*\mathcal{R})^{-1}\mathcal{S}^*\mathcal{R}\mathcal{R}^*\mathbf{x} \\ &= \mathcal{R}\mathcal{R}^*\mathbf{x} = \mathbf{x},\end{aligned}$$

$$\mathcal{H} = (\mathcal{S}^*\mathcal{R})^{-1}$$

$$\mathbf{h}_0 = \text{IDFT} \left\{ \frac{1}{\text{DFT} \{ (\mathbf{k} * \text{flip}(\mathbf{k}_{bicub})) \downarrow_{\alpha} \}} \right\}$$



R*: bicubic DS;
R: bicubic SR;
S*: current DS

Blind setting

Assume k is unknown

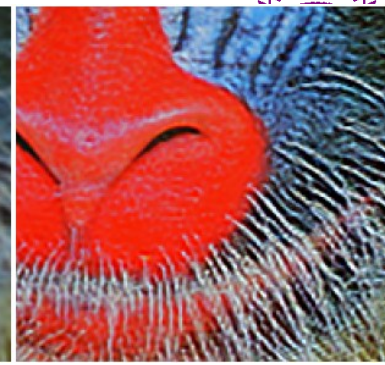
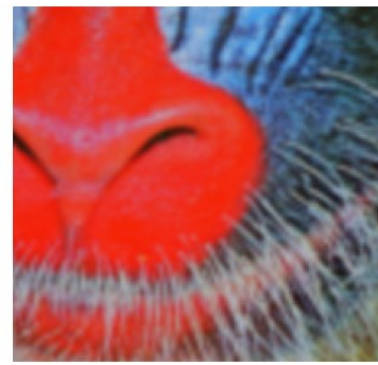
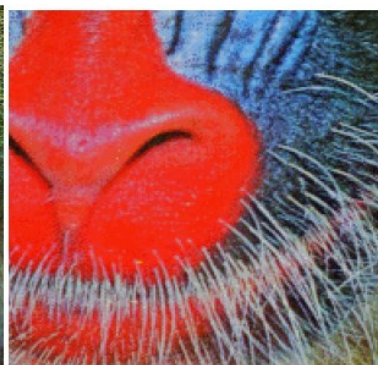
$$\|\mathbf{y} - \mathcal{S}^* \mathcal{R} \mathcal{H} \mathbf{y}\|_1$$

Optimize h

$$\|\mathcal{I} - \mathcal{H} \mathcal{S}^* \mathcal{R}\|_1 + \lambda \|\mathbf{k}\|_1$$

Optimize k

Use optimization framework
to learn h for each image



original

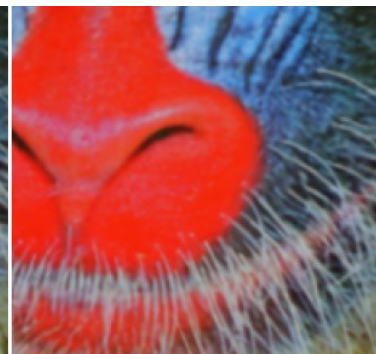
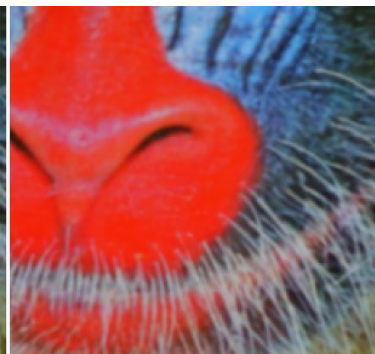
Bicubic interpolation

KernelGAN

proSR

RCAN

DBPN



Result
(blind SR)

with estimated correction



Note on data vs regularization terms

- Recap
 - Data term involves low-quality input
 - Regularization term involves human priors
- Traditional approach
 - Data term provides solution candidates
 - Regularization selects proper solutions
 - Use human priors to make results realistic
 - But limited by computable priors, e.g. loss, augmentation
- New approach with diffusion model (DM)
 - Regularization (learned DM) provides solution candidates
 - Data-driven DM generates realistic images most of the time
 - Data term, as guidance, selects proper solutions